

---

# ON THE NOTIONS OF BOUNDED BYPASS, AND HOW TO MAKE ANY DEADLOCK-FREE MUTEX PROTOCOL SATISFY ONE OF THEM

ROB VAN GLABBEEK <sup>a</sup>, DANIELE GORLA <sup>b</sup>, AND MYRTHE SPRONCK <sup>c</sup>

<sup>a</sup> School of Informatics, University of Edinburgh, UK  
School of Computer Science and Engineering, University of New South Wales, Sydney, Australia  
*e-mail address*: rvg@cs.stanford.edu

<sup>b</sup> Dept. of Computer Science, Sapienza University of Rome, IT  
*e-mail address*: gorla@di.uniroma1.it

<sup>c</sup> Dept. of Mathematics and Computer Science, Eindhoven University of Technology, The Netherlands  
*e-mail address*: m.s.c.spronck@tue.nl

---

**ABSTRACT.** In the literature on mutual exclusion, bounded bypass has been used for a long time as a strengthening of starvation-freedom, but, to the best of our knowledge, it still lacks a satisfying definition as a liveness property on its own. Moreover, we have encountered MUTEX protocols for which this notion needs to be slightly weakened in order to be met.

To solve these issues, we first provide a formal definition of bounded bypass (that also corrects a previous definition from Raynal) and then introduce the notions of *post-doorway* and *intermittent* bounded bypass, two liveness properties that lie between starvation-freedom and bounded bypass. Essentially, intermittent bounded bypass weakens bounded bypass by ignoring the possible bypasses that may happen during the execution of a certain finite set of write operations to shared registers. Orthogonally, post-doorway bounded bypass ignores the bypasses that may happen during a finite initial phase of the `lock` protocol.

Furthermore, we study an algorithm proposed by Yoah Bar-David in 1998 to enhance the liveness properties of any deadlock-free MUTEX protocol and prove that: (1) in the setting of atomic registers, this algorithm upgrades any deadlock-free mutual exclusion protocol to a bounded bypass one, with a bound that is quadratic in the number of processes; and (2) in the setting of safe and regular registers, the very same algorithm ensures the intermittent version of bounded bypass, still with a quadratic (but slightly different) bound.

Finally, we provide logical formulae for the different notions of bounded bypass defined in this paper and use them to confirm all claims made here, by using model checking. This had a positive impact on the theoretical development of the work, since it allowed us to identify and correct small mistakes/ambiguities in definitions and proofs.

---

*Key words and phrases*: Mutual exclusion, atomic registers, safe registers, liveness properties, overlapping reads and writes.

Supported by Royal Society Wolfson Fellowship RSWF\R1\221008.

We thank Bas Luttik for suggestions on the text and Tim Willemse for improvements to the modal  $\mu$ -calculus formulas.

## 1. THE MUTEX PROBLEM

The MUTual EXclusion problem (MUTEX, for short) [Dij65] is one of the most fundamental problems in concurrent and distributed systems. So, it is not surprising that many protocols have been devised in the last 60 years to solve it [Ala03, And93, Ara11, BL93, BDH16, Kes82, Knu66, Lam86a, Pet81, SWPL11, Szy88, Szy90] (just to cite a few). Its general formulation assumes a set of  $n$  ( $\geq 1$ ) processes  $p_0, p_1, \dots, p_{n-1}$  running in parallel, each sequentially executing its code. Processes are *reliable* and *asynchronous*. “Reliable” means that each process correctly executes the code of the corresponding algorithm. “Asynchronous” means that there is no assumption on the time it takes for a process to proceed from a state to the next one (which means that an asynchronous process proceeds at an arbitrary speed).

Each process has its own local memory (whose locations are denoted with lower case letters in what follows), but they can all access some shared memory to interact (whose locations are denoted with upper case letters). The memory units that a process can access are *registers*; the *domain* of a register is the set of all the values it can store (e.g., if the register is a sequence of  $k$  bits, then its domain is the set of all  $2^k$  binary strings of length  $k$ ). Every process has portions of code (called *critical sections*) that must be run in isolation to avoid inconsistencies; these usually involve writing into the shared memory.

The MUTEX problem is then formulated as follows:

**Definition 1.1.** Devise a protocol that ensures that at most one process at a time can be in a critical section.

This is usually accomplished by defining two functions: **lock**, to be run to get access to a critical section, and **unlock**, to be run when leaving the critical section.

The main correctness property of the MUTEX problem formulated above is a safety property; it is well-known that, to be meaningful, every safety property must be joint with a liveness property [AS85]. Two fundamental liveness properties for MUTEX protocols are the following:

**Definition 1.2.** A MUTEX protocol is *deadlock-free* if, whenever at some time  $t$  at least one process is running its **lock**, then afterwards a process will enter its critical section. (This needs not be one of the processes executing their **lock** at time  $t$ .)

**Definition 1.3.** A MUTEX protocol is *starvation-free* whenever every invocation of a **lock** eventually terminates (and lets the process that invoked the **lock** enter its critical section).

A third property that has been used in the literature for a long time is *bounded bypass*. A discussion of this notion and its formalization will be presented in Section 3.

## 2. ASSUMPTIONS ON THE HARDWARE

In order to reason on the correctness of MUTEX protocols, it is vital to make explicit any assumption on the hardware on which the shared memory is operating. A central assumption made by Dijkstra in the first published paper on mutual exclusion [Dij65] is the following:

*Writing a word into or nondestructively reading a word from this store are undividable operations; i.e., when two or more computers try to communicate (either for reading or for writing) simultaneously with the same common location, these communications will take place one after the other, but in an unknown order.*

This assumption is sometimes called *atomicity*. When we acknowledge (as Dijkstra did) that read and write actions take time, atomicity implies that, if process  $i$  tries to read or write to a given register while another process  $j$  is already busy reading or writing to that register, process  $i$  has to wait until process  $j$  has completed its action. Thus a read or write action of one process may be postponed by a read or write action of another; a situation that is sometimes referred to as a *blocking* [CDV09].

In most works on mutual exclusion, including [Ray13, Tau06] and this paper, it is assumed instead that reads and writes to the shared memory are *non-blocking*; if a process tries to read or write, this happens immediately. When acknowledging that reads and writes take time, it follows that different actions on the same register may overlap in time. In this setting, there exists a hierarchy of memory models, based on the values which may be returned by read actions: we say that model M1 is *at least as strong* as M2 if, for any given read, the set of values that could be returned according to M1 is contained in the one according to M2.

Two other common assumptions on the underlying hardware are that only read and write operations are available (i.e., test-and-set, compare-and-swap, or similar are not considered), and that they always terminate. According to whether a register can be written/read by just one process or by many different ones, we have: single-write/single-read (SWSR), single-write/multiple-read (SWMR), multiple-write/single-read (MWSR), or multiple-write/multiple-read (MWMR) registers. Lamport [Lam86b] proposed three memory models for SWMR registers, which form a strict hierarchy; from the weakest to the strongest, these are the *safe registers*, the *regular registers* and the *atomic registers*. In all these models, a read of a register  $R$  must always return a value from the domain of  $R$ . Whereas there are many different ways to generalize the concept of SWMR regular register to MWMR registers [SWPL11, SL23], there are canonical definitions of safe and atomic MWMR registers, which appear, e.g., in [Ray13].

**Definition 2.1.** A register  $R$  is *safe* if it guarantees the following:

- Initially, and whenever a write on  $R$  terminates when no other write on  $R$  is going on,  $R$  contains a well-defined value—the value of  $R$ —that does not change until a successive write commences.
- When a write to  $R$  of value  $v$  terminates, and this write had no overlap with any other write on  $R$ , then  $v$  becomes the value of  $R$ .
- A read that does not overlap with any write returns the value of  $R$ .

When we merely know that a register is safe, nothing more is guaranteed than the above. Thus a read that overlaps with a write may return any value in the domain of the register, and when the last of a set of partially overlapping writes terminates, the register may hold any value in its domain.

We consider regular registers only in situations where overlapping writes are excluded; this is enough since the algorithm we are going to consider in Section 4 has SWMR registers and one MWMR register that is written only in mutual exclusion mode (to be precise, at the beginning of the `unlock`). In that case, a safe register  $R$  is *regular* if it also satisfies the following requirement:

- A read that overlaps with a set of writes returns either any of the values written by these writes or the value contained in the register before the first of these writes started.

**Definition 2.2.** A register  $R$  is *atomic* if it behaves *as if*, for each read and write on  $R$ , there is a single point in time (its *execution point*) between its start and termination, when

the action takes effect. For a write action of value  $v$ , this means that  $v$  becomes the value of  $R$  at the execution point; for a read action, this means that it will return the value that  $R$  has at that time. Finally, different actions have different execution points, and the value of  $R$  changes only at the execution points of write actions.

The above definition does not stipulate that an atomic register actually features such execution points. It merely says that, for the sole purpose of determining which value may be returned by a read action, we may assume it does.

### 3. BOUNDED BYPASS

The property of *bounded bypass* (sometimes referred to as *bounded waiting*) has been used for a long time as a desirable property of MUTEX protocols. It is encountered in two variants: as a safety property<sup>1</sup> that needs to be required in conjunction with deadlock- or starvation-freedom, or as a liveness property that is intended to imply deadlock- and starvation-freedom. The former use occurs in [SGG12, Sect. 5.2] (where it is paired with a form of deadlock-freedom called *progress*), with the following definition:

*There exists a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.*

It also occurs in [Tau06, pg. 48]:

*If a given process is in its entry code, then there is a bound on the number of times any other process is able to enter its critical section before the given process does so.*

The latter use, as a liveness property that is intended to imply deadlock- and starvation-freedom, occurs in [Ray13, pg. 11], where bounded bypass is defined as

*There is a function  $f(n)$  such that, each time a process invokes **lock**, it loses at most  $f(n)$  competitions with respect to the other processes.*

with  $n$  the number of processes, and competition losing is defined as follows [Ray13, pg. 9]:

*When several processes are simultaneously executing **lock**, we say that they are competing for access to the critical section code. If one of these invocations terminates while the other invocations do not, the corresponding process  $p$  is called the winner, while each other competing process  $q$  is a loser (its invocation remains pending).*

Raynal claims that deadlock-freedom, starvation-freedom and bounded bypass (as defined above) form a “hierarchy of liveness properties: bounded bypass  $\Rightarrow$  starvation-freedom  $\equiv$  finite bypass  $\Rightarrow$  deadlock-freedom” [Ray13, pg. 11]. This is however false, since nothing guarantees that whoever invokes **lock** eventually enters its critical section in a MUTEX protocol that satisfies bounded bypass (as defined above). As an extreme example, consider a protocol in which each **lock** contains the single instruction “**await false**”.<sup>2</sup> Such a protocol trivially satisfies bounded bypass as formulated in [Ray13], since no competition will ever be lost (nor will one ever be won); yet it does not satisfy deadlock- or starvation-freedom.

<sup>1</sup>Or rather, a property that becomes a safety property when requiring a concrete bound.

<sup>2</sup>The instruction **await COND** blocks the invoking process until the boolean condition *COND* becomes true; this can be implemented, e.g., by using busy waiting, but our results are not influenced by the actual implementation.

This breaks the chain of implications reported above. Note, however, that the conjunction of deadlock-freedom and Raynal’s bounded bypass does imply starvation-freedom.

We now correct the definition of Raynal, with the intention of having one single liveness property that implies starvation-freedom (and deadlock-freedom). To this aim, we say that a process  $i$  running `lock` *loses* a competition with another process  $j$  if the `lock` of  $j$  overlaps in time with that of  $i$  and terminates earlier. A first attempt could be the following formulation:

There is a function  $f(n)$  such that, each time a process invokes `lock`, it will eventually indicate its desire to enter the critical section and, following this, it enters after losing at most  $f(n)$  competitions.

This proposal fixes two mistakes in the definition provided by [Ray13]. First, with the explicit requirement that the invoking process enters the critical section, the chain of implications reported at the end of [Ray13, Sect. 1.3.1] now holds. Second, the underlined part is missing in [Ray13], and also in [Tau06], but must be intended anyway as, without it, bounded bypass is not a useful concept. Namely, as long as a process  $p_i$  that has invoked its `lock` has not in any way communicated to the competing processes its intention to enter the critical section, from the perspective of these other processes  $p_i$  could just as well have not invoked its `lock`. For this reason, the other processes cannot hold back in favor of  $p_i$ , and so can bypass  $p_i$  as many times as they want. One might think that this issue can be mitigated by  $p_i$  indicating its desire to enter the critical section in the very first instruction of its `lock`, for instance by setting a flag that can be read by the other processes. However, even when assuming atomic registers, this first action takes time, so there exists a period during which the other processes are unaware of  $p_i$ ’s desire and can bypass  $p_i$  an unbounded number of times. To avoid this pitfall, in the spirit of the definition of Silberschatz, Galvin and Gagne [SGG12], the underlined part needs to be added to Taubenfeld’s and Raynal’s definitions, as done in our proposal.

The above formulation is however quite unsatisfactory because it leaves unspecified what it really means for a process to indicate its desire (or to make a request) to enter the critical section. In many protocols in the literature, this is quite uncontroversial: it is accomplished by setting an SWMR variable (typically called *flag*) and this usually happens at the very beginning of the `lock`. However, there are notable exceptions: the most remarkable one is Dekker’s algorithm (reported for the first time as a flow chart in [Dij62] and in pseudocode in [Ala03]), one of the first and most famous MUTEX protocols for two processes. We report it here as Algorithm 1 to ease references in what follows.

Here, the problem is that the flag (that is indeed set at the very beginning of the `lock`) may change its value during the execution of the `lock`. This illustrates our dissatisfaction with the vagueness of phrases like “indicate its desire to enter the critical section” or “made a request to enter its critical section”: which is the real moment in Dekker’s algorithm when a process indicates its desire (or made a request) to enter the critical section?

One possible answer is: the moment in which the flag does not change anymore. In this way, Dekker’s algorithm would satisfy bounded bypass. However, with this understanding, we could add to protocols an instruction that generates this very last setting just before entering the critical section; in this case, any starvation-free protocol can be turned into one that satisfies bounded bypass with bound 0!

This leads us to move to the opposite extreme and allow a process to declare its intention with the first instruction of its `lock`; we remark that this should be a write to a shared

---

**Algorithm 1** Dekker’s Algorithm (as reported in [Ala03] – it first appeared in [Dij62]). Line 1 is the preliminary initialization; lines 3 and 11 are the way in which  $p_i$  invokes **lock/unlock**, for  $i = 0$  or  $1$ .

---

```

1: Initialize FLAG[0] and FLAG[1] to 0, and TURN to either 0 or 1
2:
3: lock( $i$ ) :=
4:   FLAG[ $i$ ]  $\leftarrow$  1
5:   while FLAG[ $1-i$ ] = 1 do
6:     if TURN =  $1-i$  then
7:       FLAG[ $i$ ]  $\leftarrow$  0
8:       await TURN =  $i$ 
9:       FLAG[ $i$ ]  $\leftarrow$  1
10:
11: unlock( $i$ ) :=
12:   TURN  $\leftarrow$   $1-i$ 
13:   FLAG[ $i$ ]  $\leftarrow$  0

```

---

register, otherwise no form of bounded bypass is possible. We take the latter approach and provide our definition of bounded bypass:

**Definition 3.1.** A MUTEX protocol satisfies *bounded bypass* whenever there is a function  $f(n)$  such that every process that invokes the **lock** enters the critical section, and this happens after losing at most  $f(n)$  competitions from the moment in which it completes the execution of the first instruction of the **lock**.

Essentially, this definition provides a period of time in which many bypasses may happen (namely, the time a process takes to perform its first instruction); however, after this period, the number of possible bypasses is upper bounded by  $f(n)$ . Moreover, this aligns with virtually all MUTEX protocols found in the literature, where the desire of entering the critical section is declared in the very first instruction of their **lock**.

**Remark 3.2.** *With this definition, Dekker’s algorithm does not satisfy bounded bypass. To see this, consider the following execution:*

- assume that TURN is initialized to 0;
- $p_0$  and  $p_1$  both simultaneously do lines 4, 5 and 6;
- $p_0$  finds TURN = 0, so jumps back to line 5;
- $p_1$  finds TURN = 0, so proceeds to line 7, executes it and then sleeps for a long time;
- now, from  $p_0$ ’s point of view,  $p_1$  is like if it has not locked at all; so,  $p_0$  can enter and exit the critical section as many times as it wants (until  $p_1$  wakes up).

*In contrast, it is well known that, when assuming atomic registers, Dekker’s algorithm satisfies starvation-freedom (see [GLS25] for an automated proof of this property).*

### 3.1. Post-Doorway Bounded Bypass.

Definition 3.1 proposes a definition of bounded bypass that fixes many issues of other definitions proposed in the literature. However, we realize that this is not the only possible such definition for the intended property. We explore here a first possible variant, based

on an observation made by Lamport in [Lam86a]. Essentially, he assumes that every `lock` protocol is divided in two parts, which he calls *doorway* and *waiting*; the doorway is a portion of code whose execution requires only a bounded number of elementary operation executions (and hence always terminates). Thus, a plausible alternative to Definition 3.1 consists in counting only the bypasses that happen during the waiting part, that is, after the doorway part has been completed. This leads us to the following notion of bounded bypass, that we call *post-doorway*.<sup>3</sup>

**Definition 3.3.** A MUTEX protocol satisfies *post-doorway bounded bypass* whenever there is a function  $f(n)$  and an initial portion of the `lock` whose execution requires only a bounded number of elementary actions (the *doorway*) such that every process that invokes the `lock` enters the critical section, and this happens after losing at most  $f(n)$  competitions from the moment in which it completes the doorway.

Clearly, Definition 3.3 weakens Definition 3.1: if we have a protocol that satisfies the latter, then the protocol also satisfies the former, by letting the doorway be the very first instruction. By contrast, the reverse implication in general does not hold: imagine a `lock` protocol with a doorway made up by at least its first two instructions, run by a process that sleeps for a very long time between any of them; then, there may be no upper bound on the possible bypasses that happen during this period (that are ignored by post-doorway bounded bypass, but counted by bounded bypass).

To make this reasoning more concrete, and to show that our definition has a relevance for what has already appeared in the literature, consider Anderson’s protocol [And93], whose code is reported in Algorithm 2, to ease the following discussion.

**Remark 3.4.** *The protocol is designed for two processes only,  $p_0$  and  $p_1$ , that have different lock protocols. First of all, we want to show that Anderson’s algorithm does not satisfy Definition 3.1; to see this, consider the following execution:*

- $p_0$  executes line 4 and then sleeps for a long time;
- then,  $p_1$  executes lines 4/./7, finds its  $x$  to be 0, jumps to lines 11 and 12, enters its critical section, and unlocks;
- if now  $p_1$  invokes `lock` again, it will be exactly in the very same situation as before (since  $T[0]$  has not changed), and so will enter the critical section again; and so on, until  $p_0$  wakes up.

*This is a concrete example of what we informally discussed above, with the doorway made up of more than one instruction and one process that goes to sleep after executing the first of them.*

In contrast, we now show that Anderson’s protocol satisfies Definition 3.3; to ease reasoning, we confine ourselves to the model of atomic read and write registers.

**Proposition 3.5.** *If  $p_i$  terminates the execution of line 7 at time  $t$ , then  $p_{1-i}$  can bypass  $p_i$  at most once after time  $t$ .*

---

<sup>3</sup>[Lam86a] reports something very similar to the following definition, calling it *r-bounded waiting* (see the last paragraph of [Lam86a, Sect.2.2]) and ascribing it to Rivest and Pratt [RP76]. However, the definition in [RP76] does not mention the doorway at all; moreover, it only poses a linear bound on how many times every process may bypass a given one, whereas the following definition aligns with Raynal and assumes a global bound that can be any function of  $n$ .

---

**Algorithm 2** Anderson’s Algorithm [And93]. Line 1 is the preliminary initialization; lines 3 and 14 are the way in which  $p_0$  and  $p_1$  invoke **lock/unlock**.

---

|                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1: Initialize P[0], P[1], Q[0], Q[1], T[0] and T[1] to 1 2: 3: lock(0) := 4:   P[0] ← 0 5:   Q[0] ← 0 6:   x ← T[1] 7:   T[0] ← x 8:   if x = 1 9:   then P[0] ← 1 10:      await P[1] = 1 11:   else Q[0] ← 1 12:      await Q[1] = 1 13: 14: unlock(0) := 15:   P[0] ← 1 16:   Q[0] ← 1 </pre> | <pre> lock(1) :=   P[1] ← 0   Q[1] ← 0   x ← ¬T[0]   T[1] ← x   if x = 1   then Q[1] ← 1   await P[0] = 1   else P[1] ← 1   await Q[0] = 1 13: unlock(1) :=   P[1] ← 1   Q[1] ← 1 </pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

*Proof.* We spell out the case for  $i = 0$  and for  $T[0] = 1$  at time  $t$  (the other cases, viz. for  $T[0] = 0$  at time  $t$  and for  $i = 1$ , are handled similarly). At time  $t$ ,  $p_0$  has just finished setting  $T[0]$  to  $x$ , which is 1 in this case. It then proceeds to find 1 in  $x$  at line 8; so, it executes line 9 and arrives at line 10. Here there are two possible cases: either it finds 1 in  $P[1]$  and so it enters the critical section without bypasses, or it blocks at line 10, since  $P[1]$  is 0. In this latter case,  $p_1$  has invoked **lock** and it has not yet executed line 11 nor line 15. Let us distinguish two cases now:

- if  $p_1$  eventually arrives at line 11, it sets  $P[1]$  to 1 and suspends at line 12; this unblocks  $p_0$ , that can enter the critical section (without bypasses).
- if not, this means that  $p_1$  will eventually take (or has already taken before time  $t$ ) the **then** branch, executes lines 9 and 10, enters its critical section and then unlocks. If  $p_0$  is quick enough to react to the execution of line 15 done by  $p_1$  in its **unlock**, then  $p_0$  enters its critical section (after one bypass). Otherwise,  $p_1$  was quick enough to reinvok **lock** and execute its first line. However, since it finds 1 in  $T[0]$ , it sets its  $x$  and  $T[1]$  to 0, and so takes the **else** branch. This leads it to execute line 11 and to block at line 12; as we saw before, this is what allows  $p_0$  to enter its critical section (after one bypass now).  $\square$

To ease the previous proof, we considered the first four lines of the **lock** as the doorway. However, we believe that already after the execution of the first two lines the number of bypasses is bounded. However, the proof gets more complicated and the bound might be greater; so, there is no reason for investigating this further.

**Remark 3.6.** *To conclude the discussion on Definition 3.3, we now show that post-doorway bounded bypass strengthens starvation-freedom. By definition, every protocol that satisfies the former also satisfies the latter, whereas Dekker’s protocol is an example that satisfies the latter but not the former. To see this, we first observe that the only admissible doorway is formed by the first line of the **lock**. Indeed, the following execution shows that it is possible to execute the next instructions an unbounded number of times:*



- let  $TURN$  be 0 at the outset;
- then,  $p_0$  and  $p_1$  both simultaneously do lines 4 and 5 of the `lock`;
- $p_1$  then sleeps for an arbitrary long time before proceeding to line 6;
- $p_0$  proceeds to line 6, finds  $TURN = 0$ , and jumps back to line 5;
- $p_0$  again finds  $FLAG[1] = 1$  and  $TURN = 0$ ; so, it executes lines 5 and 6 an unbounded number of times (until  $p_1$  wakes up).

Hence, the only possible doorway is line 4 alone; this means that post-doorway bounded bypass for this protocol coincides with bounded bypass as defined in Definition 3.1. As we discussed above, Dekker’s algorithm does not satisfy Definition 3.1, so neither Definition 3.3.

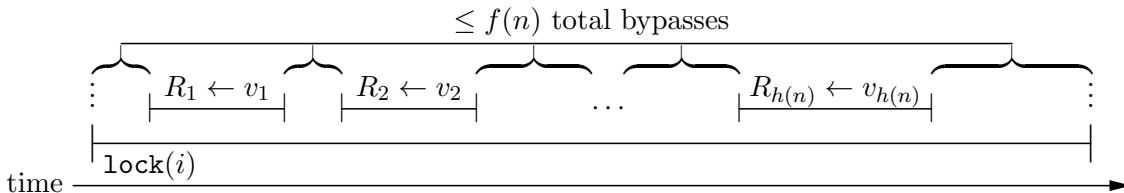
### 3.2. Intermittent Bounded Bypass.

A second alternative to Definition 3.1 is to allow arbitrary bypasses not only during the execution of the very first instruction of the `lock`, but during any finite set of (write) actions. However, these actions do not cover the period of time between the invocation of the `lock` and the starting of its first instruction (something that, in contrast, is included in the “non-counting” period of Definitions 3.1 and 3.3). Hence, to define the following notion, we say that a function (in particular, `lock`) starts when its first instruction starts. This leads to the following notion of bounded bypass, that we call *intermittent* because such “bypassing moments” can arbitrarily appear during the execution of the `lock`.

**Definition 3.7.** A MUTEX protocol satisfies an *intermittent bounded-bypass* of  $f(n)$  within  $h(n)$  interrupting assignments if a set  $S$  of assignments to shared registers can be defined, such that each time a process  $p_i$  starts its `lock`, it will reach its critical section after a period  $\pi$  such that

- at most  $h(n)$  assignments in  $S$  are made within  $\pi$ ; and
- $p_i$  enters its critical section losing at most  $f(n)$  competitions with the other processes in those parts of  $\pi$  that are outside the execution of assignments in  $S$ .

The writes in  $S$  are referred to as “interrupting assignments”. Intuitively, the intermittent bounded-bypass property weakens the bounded-bypass one by allowing a (potentially) arbitrary number of bypasses to happen during the execution of some selected assignments; indeed, bounded bypass, as in Definition 3.1, is a form of intermittent bounded bypass with one interrupting assignment, namely the first instruction of the `lock` (that should be a write to the shared memory). Unlike bounded bypass, where the only interrupting assignment is an assignment by the relevant process itself, intermittent bounded bypass allows *any* assignment to be defined as interrupting, including those performed by other processes. Pictorially, we can represent the intermittent scenario as follows:



Similar to the bounded bypass case, also here the first instruction of a `lock` must be a write to the shared memory; moreover, in this setting, it always counts as interrupting. We remark

| Protocol         | Memory model | B.B.              | Post-doorway B.B. | Intermittent B.B.               |
|------------------|--------------|-------------------|-------------------|---------------------------------|
| <i>Dekker</i>    | Atomic       | X                 |                   |                                 |
| <i>Anderson</i>  | Atomic       | X                 | $f(n) = 1$        | X                               |
| <i>Bar-David</i> | Atomic       | $f(n) = n(n-1)-1$ |                   | $f(n) = n(n-1)-1$<br>$h(n) = 1$ |
| <i>Bar-David</i> | Safe/Regular | X                 |                   | $f(n) = n^2-2$<br>$h(n) = n$    |

Table 1: Summary of the Bounded Bypass (B.B.) properties satisfied by Algorithms 1, 2 and 3 (to be presented in the next section), according to the memory models adopted (safe/regular/atomic registers). Here, ‘X’ means that the property is not satisfied and an arithmetical expression means that the property is satisfied with that bound.

that, in reality, each interrupting assignment will take a very short time, in which it is in practice quite unlikely that many bypasses happen.

Clearly, intermittent bounded bypass implies starvation-freedom. The reverse implication does *not* hold: this is proved by the execution of Dekker’s protocol given in Remark 3.2. In the next section, we shall present a concrete protocol that satisfies this notion of bounded bypass (in the setting of safe and regular registers), but not the notions of Definition 3.1 and Definition 3.3. Intermittent bounded bypass is not a generalization of post-doorway bounded bypass: the latter includes the period between the instructions of the doorway, and also allows the doorway to contain reads; the former only includes the duration of each interrupting assignment.

### 3.3. A Hierarchy of Liveness Properties.

To conclude this section, we present a hierarchy of liveness properties that includes the notions of bounded bypass devised so far and that relates them to starvation- and deadlock-freedom. This generalizes the chain of implications intended by Raynal in [Ray13, pg. 11].

**Theorem 3.8.** It holds that

$$\begin{array}{ccccc}
 & & \text{post-doorway b. b.} & \Rightarrow & \\
 \text{bounded bypass} & \Rightarrow & & \Rightarrow & \text{starvation freedom} \Rightarrow \text{deadlock freedom} \\
 & \Rightarrow & \text{intermittent b. b.} & \Rightarrow & 
 \end{array}$$

Moreover, all missing implications do not hold.

*Proof.* All implications of the Theorem hold by definition; we are left to prove that no other implication holds. (To better visualize a sample protocol that satisfies/does not satisfy a property, see Table 1.)

- It is well-known that deadlock-freedom does not imply starvation-freedom (see, e.g., [Ray13]).
- *Starvation-freedom does not imply post-doorway/intermittent bounded bypass:* Dekker’s algorithm satisfies starvation-freedom; however, it does not satisfy any form of bounded bypass (see Remark 3.6 for post-doorway bounded bypass and the execution contained in Remark 3.2 for intermittent bounded bypass).

- *Post-doorway bounded bypass does not imply bounded bypass:* Anderson’s algorithm does not satisfy Definition 3.1 (Remark 3.4), but satisfies Definition 3.3 (Proposition 3.5).
- *Intermittent bounded bypass does not imply bounded bypass:* In the next section, we will present Algorithm 3, which in the setting of safe registers does not satisfy bounded bypass (Proposition 4.7) but does satisfy intermittent bounded bypass (Theorem 4.13).
- *Intermittent bounded bypass and post-doorway bounded bypass are logically unrelated:* If a protocol satisfies intermittent bounded bypass, the  $h(n)$  write actions may not be included in the execution of a piece of code that we consider an acceptable doorway (actually, they may also be actions belonging to the execution of the `lock` protocol by different processes – see, e.g., the proof of Lemma 4.12 later on); hence, it needs not satisfy post-doorway bounded bypass. In fact, the forthcoming Algorithm 3 does not satisfy post-doorway bounded bypass (this can be obtained by a reasoning similar to Proposition 4.7).

For the converse, consider again Anderson’s protocol; it is possible that, by stopping a process after the execution of the first instruction, it can be bypassed an unbounded number of times before it executes its second instruction. This is enough to show that this protocol fails to satisfy intermittent bounded bypass.  $\square$

Of course, we can combine the two notions of bounded bypass into a fourth property, called *post-doorway intermittent bounded bypass*, that ignores the bypasses that may happen during an initial portion of the `lock` and during a bounded number of writes that may happen in successive moments. This property still strictly implies starvation-freedom (again, Dekker’s algorithm can be used to show this). However, we are not aware of any documented protocol that merely satisfies this fourth form of bounded bypass.

#### 4. ON ENHANCING THE LIVENESS PROPERTY OF ANY DEADLOCK-FREE ALGORITHM

As we have just discussed, the different forms of bounded bypass are among the strongest liveness properties for MUTEX protocols that occur in the literature. We now show that any deadlock-free MUTEX protocol, call it DLF, can be exploited to generate a new MUTEX protocol that satisfies (intermittent) bounded bypass. This idea was first presented in [Tau06, Exercise 2.34], and declared to be devised by Yoah Bar-David in 1998; it was then formally developed in [Ray13, Sect. 2.2.2], where it was proved to promote any deadlock-free protocol to a starvation-free one, in the setting of atomic registers. The proposed protocol is reported as Algorithm 3.

This protocol maintains a Boolean variable  $\text{FLAG}[i]$  for each process  $p_i$  (for  $i \in \{0, \dots, n-1\}$ ), as well as a variable  $\text{TURN}$  that can take any value in  $\{0, \dots, n-1\}$ . The variable  $\text{FLAG}[i]$  tells whether  $p_i$  wants to enter its critical section; it is set to 1 at the beginning of the `lock` function of  $p_i$ , and to 0 at the beginning of its `unlock`. The variable  $\text{TURN}$  tells which is the most privileged process to enter the critical section; its value cycles through all process IDs, and is incremented only in the `unlock`, whenever  $p_{\text{TURN}}$  has not signaled its intention to enter the critical section. For the rest, the protocol is extremely simple: a process  $p_i$  that wishes to enter the critical section waits until either  $\text{TURN}$  becomes  $i$ , or  $p_{\text{TURN}}$  has not signaled intention to enter. After this wait, the process runs the `lock` of the given protocol DLF and, naturally, the `unlock` of DLF is run to complete the protocol.

[Ray13, Tau06] work in the setting of atomic registers, for which they prove that Algorithm 3 promotes DLF to a starvation-free protocol. In the next two sections, we prove that the very same protocol (1) satisfies bounded bypass (with a bound that is quadratic

---

**Algorithm 3** Enhancing the Liveness of a deadlock-free MUTEX protocol (Bar-David 1998). Line 1 is the preliminary initialization; lines 3 and 10 are the way in which `lock/unlock` are invoked by process  $p_i$ , for  $i \in \{0, \dots, n-1\}$ . The symbolic names to read/write actions to the shared memory are used in the proofs.

---

```

1: Initialize FLAG[i] to 0, for all i, and TURN to any process ID ( $\in \{0, \dots, n-1\}$ )
2:
3: lock(i) :=
4:   FLAG[i]  $\leftarrow$  1                                 $\triangleright$  W1
5:   repeat
6:     tmp  $\leftarrow$  TURN                                 $\triangleright$  R1
7:   until (tmp = i  $\vee$  FLAG[tmp] = 0)                   $\triangleright$  R2
8:   DLF.lock(i)
9:
10: unlock(i) :=
11:   FLAG[i]  $\leftarrow$  0                                 $\triangleright$  W2
12:   tmp  $\leftarrow$  TURN                                 $\triangleright$  R3
13:   if (FLAG[tmp] = 0) then                             $\triangleright$  R4
14:     TURN  $\leftarrow$  (tmp + 1) mod n                     $\triangleright$  W3
15:   DLF.unlock(i)

```

---

in the number of processes involved) for atomic registers, and (2) promotes DLF to an intermittent bounded bypass protocol also in the (weaker) settings of safe and regular registers.

To carry out the proofs, we find it convenient to refer to the read/write actions (of shared registers) as follows:

- W1/W2/W3 are the write actions happening, respectively, in lines 4, 11 and 14;
- R1/R2/R3/R4 are the read actions happening, respectively, in lines 6, 7, 12 and 13.

#### 4.1. The Algorithm with Atomic Registers.

For atomic registers, [Ray13] proves that Algorithm 3 promotes any deadlock-free MUTEX protocol to a starvation-free one; here, we push this result further and prove that the resulting protocol enjoys bounded bypass, with a bound that is quadratic in the number of processes.

Since DLF satisfies mutual exclusion—never will multiple processes be in the critical section—so does Algorithm 3. We start by showing that Algorithm 3 is deadlock-free.

**Lemma 4.1.** If at some time  $t$  a process is running its `lock`, then afterwards a process will enter its critical section.

*Proof.* Since DLF enjoys deadlock-freedom, it suffices to prove that at least one process either is running DLF.lock at time  $t$ , or invokes DLF.lock after time  $t$ . Towards a contradiction, assume that at time  $t$  no process is running DLF.lock and no process ever will. Let  $t'(\geq t)$  be the time in which any  $p_j$  that has already completed DLF.lock at time  $t$  reaches line 15 (if no such  $p_j$  exists, then  $t'=t$ ). Let  $\text{TURN} = k$  at time  $t'$ ; then  $\text{TURN}$  will remain  $k$  forever.

Let  $p_i$  be running its `lock` at time  $t$ . By our assumption, it is not running DLF.lock and will never invoke it; this is possible only if  $p_i$  remains in the **repeat** loop forever and, in

particular, after time  $t'$ . For this to happen,  $i \neq k$  and, after time  $t'$ ,  $p_i$  reads  $\text{TURN} = k$  in R1 and  $\text{FLAG}[k] = 1$  in R2. But in that case  $p_k$  is also running its **lock** at time  $t'$ ; it finds  $\text{TURN} = k$  in R1, exits the **repeat** loop, and thus eventually invokes  $\text{DLF.lock}$ . This yields the required contradiction.  $\square$

Assume that  $n > 1$ . Since DLF is assumed to be a correct MUTEX protocol, at most one process can be executing the first segment of **unlock**, lines 11–14. Namely, two processes doing this are, from the perspective of DLF, indistinguishable from two processes in the critical section, and that has to be excluded. As a consequence, the variable  $\text{TURN}$  cannot change between R3 and R4, so that R4 actually checks whether  $\text{FLAG}[\text{TURN}] = 0$ .

**Definition 4.2.** We say that the variable  $\text{TURN}$  is *stably  $i$  at time  $t$* , notation  $\text{TURN} \equiv_t i$ , whenever at time  $t$   $\text{TURN} = i$  and no process  $j$  is running the part of **unlock**( $j$ ) prior to line 15.

**Lemma 4.3.** If  $\text{TURN} \equiv_t i$  and  $\text{FLAG}[i] = 1$  at time  $t$ , then no process different from  $p_i$  can progress from line 3 to line 8 between time  $t$  and the moment in which  $p_i$  executes W2.

*Proof.* Under the given hypotheses,  $\text{FLAG}[i]$  remains 1 and the value of  $\text{TURN}$  does not change from time  $t$  until  $p_i$  executes W2. Until then, no process other than  $i$  can progress from line 3 to line 8, since every other  $p_j$  gets blocked in its **repeat**.  $\square$

**Corollary 4.4.** If  $\text{TURN} \equiv_t i$  and  $\text{FLAG}[i] = 1$  at some time  $t$  when  $p_i$  is running its **lock**, then  $p_i$  enters the critical section after losing at most one competition with each of the other processes.

*Proof.* Since  $p_i$  is running its **lock** at time  $t$  and  $\text{TURN}$  remains  $i$ ,  $p_i$  will eventually invoke (or has already invoked)  $\text{DLF.lock}$ . Because of Lemma 4.3, a process  $j \neq i$  can enter its critical section after time  $t$  and before  $p_i$  does only if  $p_j$  invoked its **lock** before time  $t$ . No  $p_j$  can do this twice. Consequently,  $p_i$  can lose at most one competition with each of the other processes before entering its critical section.  $\square$

**Lemma 4.5.** If  $p_i$  runs **lock** at time  $t$ , then afterward the value of  $\text{TURN}$  will be incremented, and this happens after  $p_i$  loses at most  $n-1$  competitions.

*Proof.* Let  $p_j$  be the first process that finishes **lock** after time  $t$  (such a process exists because of Lemma 4.1). Right after doing this,  $p_j$  will enter its critical section and subsequently **unlock**. If within this **unlock**  $p_j$  finds  $\text{FLAG}[\text{TURN}] = 0$ , then  $\text{TURN}$  is increased. Otherwise,  $\text{TURN} \neq j$  and, by using Lemma 4.3 with the time in which  $p_j$  reaches line 15,  $p_j$  can not win again, and no other process can win twice until  $p_{\text{TURN}}$  wins the competition. When  $p_{\text{TURN}}$  wins, which is guaranteed to happen by Corollary 4.4, it increases  $\text{TURN}$  in its **unlock** phase. Hence  $p_i$  loses at most  $n-1$  competitions before the value of  $\text{TURN}$  is incremented.  $\square$

**Theorem 4.6. (Bounded bypass with atomic registers)** When a process completes W1, then it enters the critical section after losing at most  $n(n-1)-1$  competitions.

*Proof.* Say that  $p_i$  completes W1 at time  $t$ ; the worst case is when  $\text{TURN} = (i+1) \bmod n$  at time  $t$ .<sup>4</sup> By Lemma 4.5,  $p_i$  may lose at most  $n-1$  competitions before the value of  $\text{TURN}$

<sup>4</sup>It could be considered worse if  $\text{TURN} = i$  at time  $t$ , but another process  $j$  in its **unlock** sets  $\text{TURN}$  to  $(i+1) \bmod n$  in W3, say at time  $t' > t$ . In that case  $p_j$  must have read  $\text{FLAG}[i] = 0$  in R4, which must have occurred prior to time  $t$ . So no process has won the competition between  $t$  and  $t'$ , and we can simply start our reasoning from  $t'$ .

is incremented and so it may lose at most  $(n-1)^2$  competitions before `TURN` reaches the value  $i$ . In case  $p_i$  completes its `lock` before `TURN` reaches the value  $i$  we are done, so assume that  $p_i$  is still running `lock` when `TURN` reaches the value  $i$ . Right after this occurs, we reach a time  $t'$  when  $\text{TURN} \equiv_{t'} i$ ,  $\text{FLAG}[i] = 1$ , and  $p_i$  is still running its `lock`. After  $t'$ , by Corollary 4.4,  $p_i$  may lose at most one competition with each of the other processes before entering its critical section. Moreover, by Lemma 4.3, after  $t'$  it cannot lose another competition with the process that incremented the value of `TURN` to  $i$ . Thus, it loses at most  $n-2$  competitions after time  $t'$ . This brings the total losses to at most  $(n-1)^2 + (n-2) = n(n-1)-1$ .  $\square$

Note that the bound given in Theorem 4.6 is tight, in the sense that it can be achieved in some execution of Algorithm 3. To see this, let `TURN` be initialized to 1 and let us prove that  $p_0$  can be bypassed exactly  $n(n-1)-1$  times. Indeed:

- (1)  $p_0$  executes `W1`.
- (2) Processes from 2 to  $n-1$  execute the `lock` and assemble at the start of line 8 (they pass line 7 because  $\text{FLAG}[\text{TURN}]=0$ ).
- (3)  $p_1$  executes `W1`.
- (4) One by one, processes from 2 to  $n-1$  execute `DLF.lock` without contention, enter their critical sections and execute their `unlock`, without incrementing `TURN` (since  $\text{FLAG}[\text{TURN}]=1$ ); this step yields  $n-2$  bypasses.
- (5)  $p_1$  executes the remainder of its `lock`, its critical section and its `unlock` (where `TURN` is incremented, since  $\text{FLAG}[\text{TURN}]=0$ ); this yields another bypass.
- (6) We repeat steps (2)–(5) above  $n-3$  times, with  $p_{\text{TURN}}$  (for  $\text{TURN} = 2, 3, \dots, n-2$ ) in the role of  $p_1$  and processes  $\{1, \dots, n-1\} \setminus \{p_{\text{TURN}}\}$  in the roles of  $p_2, \dots, p_{n-1}$ .
- (7) We repeat steps (2)–(4) above with  $p_{n-1}$  in the role of  $p_1$  and processes  $\{1, \dots, n-2\}$  in the roles of  $p_2, \dots, p_{n-1}$ .
- (8)  $p_{n-1}$  executes the remainder of its `lock`, its critical section and line 11 of its `unlock`.
- (9) Now, processes from 1 to  $n-2$  execute the `lock` once more and assemble at the start of line 8 (again, they pass line 7 because  $\text{FLAG}(\text{TURN})=0$ ).
- (10)  $p_{n-1}$  finishes its `unlock`, by putting `TURN` to 0.
- (11) One by one, processes from 1 to  $n-2$  execute `DLF.lock` without contention, their critical sections, and the `unlock` (here, `TURN` is not incremented, since  $\text{FLAG}[\text{TURN}]=1$ ).
- (12)  $p_0$  finally executes the remainder of its `lock` and enters its critical section.

Steps (1) to (8) bring  $(n-1)(n-1)$  bypasses, whereas steps (9) to (11) bring another  $n-2$  bypasses. So, the total number of bypasses is  $(n-1)(n-1) + (n-2) = n(n-1)-1$ .

#### 4.2. The Algorithm with Safe and Regular Registers.

Let us now consider the behavior of the algorithm in the weakest possible memory model, namely the one with safe registers. First of all, notice that the elements of `FLAG` are SWMR safe registers, since  $\text{FLAG}[i]$  can only be written by  $p_i$  through `W1` and `W2`. Moreover, the write `W3` occurs in a portion of the code (lines 11–14) that, from the point of view of `DLF`, is treated equivalent to a critical section; hence, there are no overlapping writes.

**Proposition 4.7.** For Algorithm 3, no bound on the number of bypasses can be guaranteed with safe registers.

*Proof.* Consider a scenario with three processes  $p_0$ ,  $p_1$  and  $p_2$ . Assume that  $\text{TURN} = 0$  and that  $p_0$  is staying within its W1 for a very long time. During this time, assume that  $p_1$  always reads  $\text{FLAG}[0] = 1$  in its R2 (and so never reaches its line 8), whereas  $p_2$  always reads  $\text{FLAG}[0] = 0$  in its R2, proceeds to line 8, enters the critical section and reads  $\text{FLAG}[0] = 1$  in its R4 (and so does not increase  $\text{TURN}$ ). This can be repeated an unbounded number of times, thereby breaking any bound on the number of losses  $p_1$  may endure before entering its critical section.  $\square$

**Remark 4.8.** *The scenario above applies to regular registers as well, so also for regular registers Algorithm 3 does not satisfy bounded bypass.*

We will now show, when  $n > 2$ , that with safe registers Algorithm 3 satisfies an intermittent bounded bypass of  $n^2 - 2$  within  $n$  interrupting assignments. For this algorithm, the set  $S$  of interrupting assignments is made up by:

- the W1 operation of the process  $p_i$  considered, and
- those W1 operations of any other process  $p_j$  involved in the protocol for which there exists a moment during its execution when
  - (a) the W1 operation of the process  $p_i$  has been completed,
  - (b)  $\text{lock}(i)$  has not yet been completed, and
  - (c) the register  $\text{TURN}$  is not being written and its value is  $j$ .

By comparison, the previous section showed that, with atomic registers, Algorithm 3 satisfies an intermittent bounded bypass of  $n(n-1)-1$  within 1 interrupting assignment, namely the W1 executed by the process considered. That assignment is the one referred to in the last part of Definition 3.1, which we no longer need in Definition 3.7.

First note that Lemma 4.1 still holds for safe registers. The reason is that its proof nowhere depends on the reading of a shared register that could be written by another process at the same time.

We now write  $\text{FLAG}[i] \doteq_t 1$  if, at time  $t$ ,  $\text{FLAG}[i] = 1$  and  $p_i$  is not executing W1 or W2. If in Lemma 4.3 and Corollary 4.4 we replace  $\text{FLAG}[i] = 1$  with  $\text{FLAG}[i] \doteq_t 1$ , then these results hold also for safe registers, for the same reason as above.

**Definition 4.9.** Given any time  $t$ , let  $\hat{t}$  be the first time  $\geq t$  in which every process that has finished its  $\text{lock}$  at time  $t$  has reached line 15.

**Lemma 4.10.** For every  $t$ , no process can finish its  $\text{lock}$  between  $t$  and  $\hat{t}$ .

*Proof.* If at time  $t$  all processes that finished their  $\text{lock}$  have also already reached line 15, then  $\hat{t} = t$ . Otherwise, because of mutual exclusion of  $\text{DLF.lock}$ , there is one process that at time  $t$  finished its  $\text{lock}$  but has not yet reached line 15; so, still because of mutual exclusion, no other process can finish its  $\text{lock}$  between time  $t$  and  $\hat{t}$ .  $\square$

**Lemma 4.11.** If at time  $t$  process  $p_i$  is running  $\text{lock}$  and has already completed its W1 operation, then afterward the value of  $\text{TURN}$  will be incremented, and this happens after  $p_i$  loses at most  $n$  competitions, excluding the competitions lost during the execution of an interrupting W1 by  $p_{\text{TURN}}$ .

*Proof.* In case  $\text{TURN}$  gets incremented after time  $t$  and before any process finishes  $\text{lock}$ , there is nothing to prove. Thus, assume that this is not the case. Let  $p_j$  be the first process that finishes  $\text{lock}$  after time  $t$  (such a process exists because of Lemma 4.1). Right after doing this,  $p_j$  will enter its critical section and subsequently unlock. Let  $t'$  be the time when

$p_j$  completes the execution of R4 in its **unlock**. If within this **unlock**  $p_j$  finds  $\text{FLAG}[\text{TURN}] = 0$ , then  $\text{TURN}$  is incremented and the lemma is proved.

Otherwise,  $\text{TURN} \neq j$  and at time  $t'$  process  $p_{\text{TURN}}$  either is executing its W1 or has already done so. Let  $u_{\text{TURN}}$  and  $u'_{\text{TURN}}$  be the start and termination times of this execution of W1 by  $p_{\text{TURN}}$ , and let  $t_{\text{TURN}} = \max\{t', u_{\text{TURN}}\}$  and  $t'_{\text{TURN}} = \max\{t', u'_{\text{TURN}}\}$ . Between  $t$  and  $t_{\text{TURN}}$   $p_i$  loses at most 1 competition, namely with  $p_j$ , in case  $j \neq i$ ; indeed, the only way for  $p_j$  to find  $\text{FLAG}[\text{TURN}] \neq 0$  is to complete R4 after  $u_{\text{TURN}}$ , so  $t_{\text{TURN}} = t'$ . After  $t'_{\text{TURN}}$ , by Corollary 4.4  $p_{\text{TURN}}$  will enter the critical section after losing at most one competition with each of the other processes. When this happens,  $p_{\text{TURN}}$  increases  $\text{TURN}$  in its **unlock** phase. Hence  $p_i$  loses at most  $n-1$  competitions after  $t'_{\text{TURN}}$  and before the value of  $\text{TURN}$  is incremented. Thus, the total number of losses excluding the ones that happen between  $t_{\text{TURN}}$  and  $t'_{\text{TURN}}$  is  $n$ .

In the special case that  $j = i$ , process  $p_i$  suffers no losses at all before  $\text{TURN}$  gets incremented, and we are done. Otherwise, at time  $t'$  the W1 operation of the process  $p_i$  has been completed, but  $\text{lock}(i)$  has not yet been completed. Moreover, at time  $t'$  the value of  $\text{TURN}$  is not being written. If at that moment the W1 of  $p_{\text{TURN}}$  is still going on, then that W1 counts as an interrupting assignment. Moreover,  $t' < u'_{\text{TURN}}$ , and the interval between  $t_{\text{TURN}}$  and  $t'_{\text{TURN}}$  is fully contained in the execution of that W1. Otherwise  $t' \geq u'_{\text{TURN}}$  and the interval between  $t_{\text{TURN}}$  and  $t'_{\text{TURN}}$  is empty.  $\square$

**Lemma 4.12.** During an execution of  $\text{lock}(i)$ , at most  $n$  interrupting assignments can occur.

*Proof.* Recall that for this protocol we defined an interrupting assignment as the W1 operation of  $p_i$ , and those W1 operations of any process  $p_j \neq p_i$  for which there exists a moment during its execution when (a) the W1 operation of the process  $p_i$  has been completed, (b)  $\text{lock}(i)$  has not yet been completed, and (c) the register  $\text{TURN}$  is not being written and its value is  $j$ . Let  $t$  be the time at which  $p_i$  completes its W1 instruction. We have to show that, after  $t$  and before  $\text{lock}(i)$  terminates, at most  $n-1$  interrupting assignments occur—including interrupting assignments that start before time  $t$ , or end after  $\text{lock}(i)$  terminates.

If, at time  $t_i > t$ ,  $\text{TURN}$  ever reaches the value  $i$ , by some process completing W3, it will not be incremented further until  $p_i$  has completed its **lock**. If  $p_j \neq p_i$  executes an interrupting assignment, then there is a time  $t_j > t$  when  $p_j$  is executing its W1 operation,  $\text{lock}(i)$  has not yet been completed and the register  $\text{TURN}$  is not being written and its value is  $j$ . Hence in  $\text{unlock}(j)$ , if it ever occurs, if the value of  $\text{TURN}$  still is  $j$ , it will be incremented. Thus, each  $p_j \neq p_i$  can execute at most one interrupting assignment.  $\square$

**Theorem 4.13. (Intermittent bounded bypass with safe registers)** Algorithm 3 satisfies an intermittent bounded bypass of  $n^2-2$  within  $n$  interrupting assignments.

*Proof.* Say that  $p_i$  invokes  $\text{lock}(i)$  at time  $t_0$  and completes W1 at time  $t > t_0$ ; by Lemma 4.10, no process will finish its **lock** between  $t$  and  $\hat{t}$ . The worst case is when  $\text{TURN} = (i+1) \bmod n$  at time  $\hat{t}$ . By Lemma 4.11,  $p_i$  may lose at most  $n$  competitions within 1 interrupting assignment before the value of  $\text{TURN}$  is incremented, and so it may lose at most  $(n-1)n$  competitions within  $n-1$  interrupting assignments before  $\text{TURN}$  reaches the value  $i$ . In case  $p_i$  completes its **lock** before  $\text{TURN}$  reaches the value  $i$  we are done, so assume that  $p_i$  is still running **lock** when  $\text{TURN}$  reaches the value  $i$ . Right after this occurs, we reach a time  $t'$  when  $\text{TURN} \equiv_{\text{mod } n} i$  and  $p_i$  is still running its **lock**.

Since  $p_i$  has already completed W1 at time  $t < t'$ ,  $\text{FLAG}[i] \doteq_{\text{mod } n} 1$ . Thus, by Corollary 4.4, after  $t'$   $p_i$  may lose at most one competition with each of the other processes. Moreover, by



Lemma 4.3, after  $t'$  it cannot lose another competition with the process that incremented the value of `TURN` to  $i$ . Thus, it loses at most  $n-2$  competitions after time  $t'$ . This brings the total losses to at most  $(n-1)n + (n-2) = n^2-2$ , within  $n$  interrupting assignments (including the execution of `W1` between  $t_0$  and  $t$ ).  $\square$

Also in this case, the bound given in Theorem 4.13 is tight, at least when  $n > 2$ . To see this, let `TURN` be initialized to 1 and let us prove that  $p_0$  can be bypassed exactly  $n^2-2$  times. Indeed:

- (1)  $p_0$  invokes `lock` and executes its `W1`.
- (2) Let  $p$  be any process in  $\{p_1, \dots, p_{n-1}\}$  different from  $p_{\text{TURN}}$  and let it invoke the `lock`, complete it (since it reads `FLAG[TURN]=0`) and execute its critical section.
- (3)  $p_{\text{TURN}}$  invokes `lock` and starts `W1`.
- (4)  $p$  executes `unlock` without changing `TURN` (indeed, it may read 1 from `FLAG[TURN]`).
- (5) All processes in  $\{p_1, \dots, p_{n-1}\}$  different from  $p_{\text{TURN}}$  now invoke the `lock` and assemble at line 8 (since they can all read 0 from `FLAG[TURN]`).
- (6)  $p_{\text{TURN}}$  completes `W1`.
- (7) All processes in  $\{p_1, \dots, p_{n-1}\}$  different from  $p_{\text{TURN}}$  one by one execute `DLF.lock` without contention, enter their critical sections and execute the `unlock` without changing `TURN` (indeed, they all read 1 from `FLAG[TURN]`).
- (8)  $p_{\text{TURN}}$  completes the `lock`, executes its critical section and the `unlock`, where it increments `TURN`. The bypasses undergone by  $p_0$  so far are  $n$ .
- (9) Now, repeat steps (2)–(8) again, for `TURN` = 2, 3,  $\dots$ ,  $n-2$ .
- (10) Finally, repeat steps (2)–(7) for `TURN` =  $n-1$ ; then, let  $p_{n-1}$  complete the `lock`, execute its critical section and the first line of the `unlock`.
- (11) All processes in  $\{p_1, \dots, p_{n-2}\}$  now invoke the `lock` and assemble at line 8 (since they all read 0 from `FLAG[TURN]`).
- (12)  $p_{n-1}$  then completes the `unlock`, by setting `TURN` to 0.
- (13) All processes in  $\{p_1, \dots, p_{n-2}\}$  one by one execute `DLF.lock` without contention, enter their critical section and execute the `unlock` without changing `TURN`.
- (14) To conclude,  $p_0$  enters its critical section.

Steps (2) to (10) bring  $n(n-1)$  bypasses, whereas steps (11) to (13) bring another  $n-2$  bypasses. So, the total number of bypasses is  $n(n-1) + (n-2) = n^2-2$ .

Notice that the above argument does not go through when  $n = 2$ , as in step (2) no  $p$  with the required properties can be found. In this case, only 1 bypass is possible, with only 1 interrupting assignment (namely, the `W1` operation of the bypassed process). This has been confirmed by means of model checking—see Section 5.

**Remark 4.14.** *Since regular registers allow only a subset of the possible behaviors of safe registers, trivially Theorem 4.13 also applies to regular registers. Moreover, the execution witnessing that the bound is tight applies easily to regular registers as well, since the only overlaps are on Boolean registers (flags) and they are only written to change the value; safe and regular registers coincide in this case.*

## 5. MODEL CHECKING

In [GLS25] the mutual exclusion and starvation-freedom properties of Algorithms 1, 2 and 3 have been automatically verified using the model checker of the mCRL2 toolset [BGK<sup>+</sup>19].

Notably, Dekker’s and Anderson’s algorithms are found to satisfy mutual exclusion and starvation-freedom with atomic registers. In the case of Algorithm 3, Lamport’s One-Bit protocol is used as DLF. The resulting protocol, verified up to 3 processes, is found to behave as well as Lamport’s Three-Bit protocol [Lam86a], as it satisfies starvation-freedom for all the three models considered in this paper (safe/regular/atomic registers).

The various forms of bounded bypass we presented in this paper have not been formalized in [GLS25], but we do this here. In order to reapply the work of [GLS25], all we need to do is to capture these properties as formulae in the version of the modal  $\mu$ -calculus [Pra81, Koz83] employed by mCRL2 (that allows recursion variables to be parametrised by higher-order data – such as numbers, sets and functions), and make minor changes to the models to support these formulae. Of note is that mCRL2 models have a single initial state. When verifying algorithms with multiple initial states, these being Dekker’s and Bar-David’s algorithms due to the different possible initial values of TURN, we check every initial state separately.

In Section 3, it was stated that bounded bypass can be viewed as a safety property that is required in conjunction with deadlock- and starvation-freedom, or as a liveness property that implies deadlock- and starvation freedom. Throughout this paper, the latter interpretation is used. However, for formalizing the properties it is much simpler to only consider the safety aspect. Deadlock- and starvation-freedom are formalized and verified separately in [GLS25]; thus it is sufficient to only check the safety aspect here.

We now provide a high-level description of the formulae for the different notions of bounded bypass; their concrete syntax in mCRL2, as well as the mCRL2 models of the algorithms with the different register types, are available online at Zenodo [GGS26]. We also report on the results of checking these formulae with mCRL2. To do this, we follow the approach in [GLS25] and run the model checker only for 2 or 3 processes; going beyond this limit would make the checking practically unfeasible due to state-space explosion.

Below, we write  $\mathbb{P} = \{p_0, \dots, p_{n-1}\}$  for the set of  $n$  processes participating in a mutual exclusion protocol. We model such a protocol as a labeled transition system that features the actions  $sl_p$  and  $fl_p$ , stating that process  $p \in \mathbb{P}$  starts and finishes its `lock`. Notice that finishing `lock` in this model coincides with entering into the critical section; hence, actions  $fl_p$  will be used to count the bypasses. We write  $fl_{all}$  for the set  $\{fl_q \mid q \in \mathbb{P}\}$ , the finish-lock actions of all processes, and  $fl_p$  for  $\{fl_q \mid q \in \mathbb{P} \wedge q \neq p\}$ , the finish-lock actions of all processes except  $p$ . We use  $sw_p$  and  $fw_p$  for signaling when a process  $p \in \mathbb{P}$  starts and finishes a write operation, respectively; we define  $sw_{all}$  and  $fw_{all}$  analogously to  $fl_{all}$ . Finally, we write  $acts$  for the set of all actions in the model, with  $acts_p$  being all actions by process  $p$  and  $acts_p$  being all actions by processes other than  $p$ .

### 5.1. Bounded Bypass and Post-Doorway Bounded Bypass.

A modal  $\mu$ -calculus formula partially expressing post-doorway bounded bypass is obtained by adapting the bounded bypass formula presented in [HKG21]. To this aim, we add to our model a special action  $doorway_p$ , stating that process  $p$  leaves the doorway section of its lock. In fact, this action is implemented as an extra tag on the final action of the doorway.

The formula  $pdbb_p(b)$  defined below intuitively expresses that each time process  $p \in \mathbb{P}$  invokes the `lock`, it finishes the `lock` after losing at most  $b$  competitions from the moment in which it performs the action  $doorway_p$ . To find out whether a given protocol satisfies post-doorway bounded bypass, for a given  $n$ , a given bound  $f(n)$ , and a given placement of

the doorway, we thus have to check the formula  $\bigwedge_{p \in \mathbb{P}} pdbb_p(f(n))$ . This formula expresses post-doorway bounded bypass only partially, because it does not quantify over all valid placements of the doorway in the protocol, and does not check whether executing the doorway requires only a bounded number of actions. So, a thorough check involves checking this formula for versions of the  $n$ -process protocol for each reasonable placement of the doorway.

Formally, we let

$$\begin{aligned} pddb_p(b) &:= [acts^*][sl_p][\overline{\{doorway_p\}^*}][doorway_p]count\_bypasses_p(b) \\ count\_bypasses_p(b) &:= \nu X(b).(\overline{fl_{all}}X(b) \wedge [fl_p](b > 0 \wedge X(b-1))) \end{aligned}$$

The formula  $[acts^*]\psi$  says that condition  $\psi$  must hold after any sequence of actions, and thus in any reachable state of the protocol. Likewise  $[sl_p]\phi$  says that condition  $\phi$  must hold in any state reachable by a transition labeled  $sl_p$ . So together the formula  $pddb_p(b) = [acts^*][sl_p]\phi$  says that some condition  $\phi$  must hold after any execution ending with the start of  $p$ 's **lock**. This condition  $\phi$ , in turn, says that the formula  $count\_bypasses_p(b)$  must hold after any sequence of actions ending with the first occurrence of the action  $doorway_p$ . Here  $\overline{S}$  denotes the complement (w.r.t.  $acts$ ) of a set  $S \subseteq acts$ . Finally,  $count\_bypasses_p(b)$  tells us that process  $p$  loses at most  $b$  competitions. It is expressed by means of a largest fixed point construct:  $count\_bypasses_p(b)$  holds in all states such that

- after any action other than  $fl_q$  for some  $q \in \mathbb{P}$ , the same condition continues to hold, and
- after any bypass by another process (i.e., after some  $fl_q$  for some  $q \neq p$ ), the same condition holds, but now with a budget  $b-1$  of one bypass less. Moreover, the formula evaluates to *false* in case a bypass occurs when  $b = 0$ .

For understanding the formula, it may help to add the conjunct  $[fl_p]true$ , saying that when process  $p$  finishes its **lock**, there are no further requirements to check; however, this conjunct is vacuously true, so need not be included explicitly.

The formula for bounded bypass is exactly the same, but now fixing  $doorway_p$  to coincide with the end of the first instruction of  $p$ 's **lock**.

Using this formula, we can confirm the results on bounded bypass and post-doorway bounded bypass reported in this paper for Dekker's and Anderson's algorithms. For Dekker's, we place the *doorway* tag at the end of the first line (as we discussed in Remark 3.6, this is the only possible placement of the doorway). It is not (currently) possible with mCRL2 to check that  $pddb_p(b)$  is violated for all natural numbers  $b$ . However, for any bound we fill in (up to 100), bounded bypass is violated; moreover, we can replay in the tool the execution that witnesses that an arbitrary number of bypasses is possible (shown in Remark 3.2). For Anderson's algorithm, we place the *doorway* tag at the end of the fourth line and find that a bound of 1 is satisfied, but a bound of 0 is violated. We also find that the doorway can already end after the second line, but in that case a bound of 2 is needed.

We also confirm the result of Theorem 4.6 for 2 and 3 processes: for both  $n = 2$  and  $n = 3$ , we find a tight bound of  $n(n-1)-1$  bypasses for a process that completes W1. Actually, in a preliminary version of this paper, we proved a  $n(n-1)$  bound for Algorithm 3 in the setting of atomic registers; it was the model checker that pointed out to us that such a bound was not tight and led us to reconsider our proofs, to tightly set the bound of Theorem 4.6 at  $n(n-1)-1$ .

The result of Theorem 4.6 applies when atomic registers are used. As is stated in Proposition 4.7 and Remark 4.8, this bound does not apply when safe or regular registers

are used instead. Using the formula, we confirm that in those cases, no bound up to 100 is satisfied when there are 3 processes. For the specific case of 2 processes, we find that with both types of register a bound of 1 is sufficient. This is in accordance with the claim above Remark 4.14.

## 5.2. Intermittent Bounded Bypass.

To capture intermittent bounded bypass, we need to make Definition 3.7 completely unambiguous, by deciding which assignments count as interrupting. Naturally, an interrupting assignment may start before the process  $p_i$  invokes its `lock`, or end after  $p_i$  reaches the corresponding critical section, as long as it overlaps with  $p_i$ 's `lock`. The difficulty lays in the quantification over all acceptable definitions of the specific interrupting assignments. Following the application in the case study of Section 4, whether a write counts as an interrupting assignment may depend on the particular run in which it occurs. Here we opt for allowing *any* choice of which writes in a run are classified as interrupting or non-interrupting.

We use the variables  $b \in \mathbb{N}$  and  $h \in \mathbb{N}$  to store the remaining budget of bypasses and interrupting assignments, respectively. Furthermore  $W \subseteq \mathbb{P}$  collects those processes that are currently engaged in a write operation. The formula  $ibb_p(h, b, W)$ , defined below, holds in a given state of the mutual exclusion protocol with active writes  $W$ , when the process  $p \in \mathbb{P}$  will, in each possible execution continuing from that state, after any given occurrence of the action  $sl_p$ , execute the action  $fl_p$  after experiencing no more than  $b$  bypasses, not counting those that occurred during a collection of no more than  $h$  writes. Thus, a mutual exclusion protocol satisfies intermittent bounded bypass of  $f(n)$  within  $h(n)$  interrupting assignments, for  $n = |\mathbb{P}|$ , when the initial state satisfies the formula  $\bigwedge_{p \in \mathbb{P}} ibb_p(h(n), f(n), \emptyset)$ .

The formula  $ibb_p(h, b, W)$  is defined in four steps. In the first step,  $ibb_p(h, b, W)$  is defined in terms of an auxiliary formula  $pre\_count_p(h, b, W)$ , which describes the intended behavior after the crucial action  $sl_p$  has occurred:

$$\begin{aligned} ibb_p(h, b, W) := \nu X(h, b, W) . ( & \bigwedge_{q \in \mathbb{P}} [sw_q] X(h, b, W \cup \{q\}) \\ & \wedge \bigwedge_{q \in \mathbb{P}} [fw_q] X(h, b, W \setminus \{q\}) \\ & \wedge [sw_{all} \cup fw_{all}] X(h, b, W) \\ & \wedge [sl_p] pre\_count_p(h, b, W)) \end{aligned}$$

In particular,  $ibb_p(h, b, W)$  holds for the largest set of states such that

- after the action  $sl_p$  occurs, the formula  $pre\_count_p(h, b, W)$  should hold, and
- after any action  $a \in acts$  occurs, the same formula  $ibb_p(h, b, W')$  should still hold, where  $W'$  is an updated version of  $W$ , by adding or removing a process from  $W$  in case  $a$  is the start or finish of a write operation.

The second bullet point applies also when  $a = sl_p$ , because  $pre\_count_p(h, b, W)$  should hold after *any* occurrence of  $sl_p$ , not merely after the first one.

Above we defined the `lock` to start when its first instruction starts, but since in our model the start of the `lock` is captured by the special actions  $sl_p$ , our model creates space between the start of the lock and its first instruction. Hence, in our formula we explicitly have to skip over this gap before we start counting bypasses. This is the function of the second step in our definition of  $ibb_p(h, b, W)$ , which defines  $pre\_count_p(h, b, W)$  in terms of  $skip\_first(h, b, W)$ , which is like  $pre\_count_p(h, b, W)$ , but starting at the first action of `lock`:

$$\begin{aligned}
pre\_count_p(h, b, W) := \nu X(h, b, W) . (h > 0 \wedge & \bigwedge_{q \in \mathbb{P} \setminus \{p\}} [sw_q] X(h, b, W \cup \{q\}) \\
& \wedge \bigwedge_{q \in \mathbb{P} \setminus \{p\}} [fw_q] X(h, b, W \setminus \{q\}) \\
& \wedge [acts_p \setminus (sw_{all} \cup fw_{all})] X(h, b, W) \\
& \wedge [acts_p \setminus sw_p] false \\
& \wedge [sw_p] skip\_first(h-1, b, W \cup \{p\}))
\end{aligned}$$

From our analysis we know that no protocol can satisfy intermittent bounded bypass, unless the first action of the lock is a write, and this write counts as interrupting. For this reason, our formula  $pre\_count_p(h, b, W)$  evaluates to *false* in case  $h = 0$ , and when the first action of  $p$  encountered by  $pre\_count_p(h, b, W)$  is anything other than a write. Moreover, when encountering this first write we decrement  $h$  by 1.

The third step in our definition defines  $skip\_first_p$  in terms of  $count\_bypasses_p$  in a similar vein, by not counting any bypasses until we reach the end of the first (write) action of  $p$ .

$$\begin{aligned}
skip\_first(h, b, W) := \nu X(h, b, W) . ( & \bigwedge_{q \in \mathbb{P} \setminus \{p\}} [sw_q] X(h, b, W \cup \{q\}) \\
& \wedge \bigwedge_{q \in \mathbb{P} \setminus \{p\}} [fw_q] X(h, b, W \setminus \{q\}) \\
& \wedge [sw_{all} \cup fw_{all}] X(h, b, W) \\
& \wedge [fw_p] count\_bypasses_p(init(h, b, W \setminus \{p\})))
\end{aligned}$$

where  $init(h, b, W)$  will be described in a moment.

The formula  $count\_bypasses_p$  keeps an open mind on whether any ongoing write shall be counted as interrupting or not. To this end it maintains as a parameter not just a single triple  $(h, b, W)$  as used above, but a set  $O$  of triples  $(h, b, I)$ , with  $I$  denoting those writes that are currently active and are deemed interrupting. For each possible classification of encountered writes as interrupting or non-interrupting, we maintain such an option  $(h, b, I) \in O$ . The formula  $count\_bypasses_p(O)$  expresses that process  $p \in \mathbb{P}$  will, in each possible execution continuing from the current state, for some  $(h, b, I) \in O$ , execute the action  $fl_p$  after experiencing no more than  $b$  bypasses, not counting those that occurred during a collection of at most  $h$  writes chosen to be classified as interrupting, including the set  $I$  of ongoing writes:

$$\begin{aligned}
count\_bypasses_p(O) := \nu X(O) . (O \neq \emptyset \wedge & \bigwedge_{q \in \mathbb{P}} [sw_q] X(wstart(O, q)) \\
& \wedge \bigwedge_{q \in \mathbb{P}} [fw_q] X(wfinish(O, q)) \\
& \wedge [fl_p] X(bypass(O)) \\
& \wedge [sw_{all} \cup fw_{all} \cup fl_{all}] X(O))
\end{aligned}$$

Here we use the following functions:

$$\begin{aligned}
init(h, b, W) &:= opt(\{(h - |I|, b, I) \mid I \subseteq W \wedge h - |I| \geq 0\}) \\
wstart(O, q) &:= opt(O \cup \{(h - 1, b, I \cup \{q\}) \mid (h, b, I) \in O \wedge h > 0\}) \\
wfinish(O, q) &:= opt(\{(h, b, I \setminus \{q\}) \mid (h, b, I) \in O\}) \\
bypass(O) &:= opt(\{(h, b, I) \mid (h, b, I) \in O \wedge I \neq \emptyset\} \cup \{(h, b - 1, \emptyset) \mid (h, b, \emptyset) \in O \wedge b > 0\})
\end{aligned}$$

where

$$opt(O) := \{(h, b, I) \in O \mid \neg \exists (h', b', I') \in O. (h \leq h' \wedge b \leq b' \wedge I \subseteq I' \wedge (h', b', I') \neq (h, b, I))\}$$

When  $count\_bypasses_p$  is called, one option  $(h, b, I) \in O$  is created for each choice of a set  $I \subseteq W$  of the ongoing writes as being interrupting; this is captured by the function  $init$ . Each time we encounter a start-write, say by process  $q \in \mathbb{P}$ , we double the amount of options by allowing this write to be interrupting or not, unless we have exhausted our budget of interrupting assignments. When we encounter a bypass (an action in  $fl_p$ ), we decrement the

bound  $b$  only for those options  $(h, b, I) \in O$  in which no interrupting assignment is active, that is,  $I = \emptyset$ . Such an option is deleted in case the budget  $b$  is already exhausted. Finally, in case any action occurs that is not a start or finish write, or a finish-lock, we simply keep checking the same formula. The first clause  $O \neq \emptyset$  of  $\text{count\_bypasses}_p(O)$  says that the formula evaluates to *false* when we run out of valid options.

To improve performance when model checking, we call the function *opt* whenever we create a new set  $O$ , which removes non-optimal options from the set; an option is non-optimal when another option in the set is strictly better in one dimension (more budget for interrupting assignments, more budget for bypasses, or a superset of the set of interrupting assignments) and at least as good in the others.

We use this formula to check the claims made in Section 4.2 for Algorithm 3 with Lamport’s One-Bit protocol as DLF.

With safe registers, we find that the bound of  $n^2 - 2$  bypasses within  $n$  interrupting assignments reported in Theorem 4.13 is indeed satisfied and tight for 3 processes. With 2 processes both the bound on bypasses and the bound on interrupting assignments can be decremented by one, in accordance with our observations at the end of Section 4 and in Section 5.1 above.

With regular registers, the results are exactly the same as with safe ones: the bound of  $n^2 - 2$  bypasses within  $n$  interrupting assignments is satisfied and tight for 3 processes, while the bound can be lowered by 1 in both directions for 2 processes.

As is noted in Section 3.2, intermittent bounded bypass coincides with bounded bypass when there is a bound on interrupting assignments of exactly 1, namely the first instruction of the **lock**. As such, we can also run the intermittent bounded bypass formula on the model with atomic registers, using a bound of 1 for interrupting assignments and of  $n(n - 1) - 1$  for bypasses. We find that these bounds are satisfied and tight for both 2 and 3 processes.

To verify the correctness of the proof of Theorem 4.13, we also developed an alternative formula for intermittent bounded bypass that specifically classifies as interrupting those writes that are treated as such in that proof, namely the ones indicated in the text below Remark 4.8—this is different from the approach presented above, where every possible choice of at most  $h(n)$  interrupting assignments is allowed. That formula is included in [GGS26] for the interested reader, but not here. When checked on the safe register model, it confirms the bounds reported in Theorem 4.13, and as such confirms that these writes are indeed the  $h(n)$  interrupting assignments that are needed for this proof.

The process of developing this formula helped us refine the definition of which writes should be counted as interrupting in Algorithm 3. In an earlier proposal, instead of requiring conditions (a), (b) and (c) to be satisfied at some moment *during* the execution of the write, it was required that they be satisfied at the *end* of the write. In the corresponding earlier version of our alternative formula, this resulted in too few writes being classified as interrupting, and hence the bound on bypasses being violated. The violating traces produced by the model checker for this version of the property helped identify and fix the issue.

## 6. DISCUSSION AND CONCLUSIONS

In this paper, we provided a few notions of bounded bypass as liveness properties that, as Raynal advocates, strengthen the notion of starvation-freedom. In doing this, we corrected two mistakes in the definition provided in [Ray13] by explicitly requiring that: (1) the process that invokes **lock** eventually enters the critical section, and (2) this must happen

after a bounded number of bypasses *after the moment in which the process has completed the execution of the first instruction of its lock*. Without the first requirement, bounded bypass does not imply starvation-freedom; without the second amendment, we argued that no MUTEX protocol satisfies bounded bypass.

Next, we presented two weakenings of bounded bypass, namely post-doorway and intermittent bounded bypass, that are half-way properties between starvation-freedom and bounded bypass. Subsequently, we proved that every deadlock-free protocol can be turned into one that satisfies (some form of) bounded bypass, both in the setting of safe, regular and atomic registers.

In addition to developing new definitions, we also prove that they are (or are not) satisfied for three relevant algorithms. Building on this, we take a lesson from Lamport, who stated in [Lam86a]: ‘*Our proofs have been done in the style of standard “journal mathematics”, using informal reasoning that in principle can be reduced to very formal logic, but in practice never is. [...] The behavioral reasoning used in our correctness proofs, and in most other published correctness proofs of concurrent algorithms, is inherently unreliable; we advise the reader to be skeptical of such proofs.*’ Thus, to support the abstract reasoning provided in the proofs, we formalized the definitions in a formal logic and used model checking to confirm our claimed results. We thereby aim to reduce reader’s skepticism and improve developer’s confidence.

Of particular note is our verification of Algorithm 3. The starvation-freedom of this algorithm was automatically verified in [GLS25], by using Lamport’s One-Bit protocol [Lam86a] as DLF and by running the resulting code for 3 processes. There, it was shown that the resulting protocol behaves as well as Lamport’s Three-Bit protocol [Lam86a], as it satisfies starvation-freedom for all the three models considered in this paper (safe/regular/atomic registers). Here, we went one step further: we provided formulae for the different types of bounded bypass defined in this paper and used them to confirm all claims made here. This also had a positive impact on the theoretical development of the work, since it identified for the protocol of Algorithm 3 a non-tight bound (in the setting of atomic registers) and an imprecise identification of the interrupting assignments (in the setting of safe registers); this allowed us to fine-tune the theoretical part accordingly.

## REFERENCES

- [Ala03] K. Alagarsamy. Some myths about famous mutual exclusion algorithms. *SIGACT News*, 34(3):94–103, 2003. doi:10.1145/945526.945527.
- [And93] James H. Anderson. A fine-grained solution to the mutual exclusion problem. *Acta Informatica*, 30:249–265, 1993. doi:10.1007/BF01179373.
- [Ara11] Alex A. Aravind. Yet another simple solution for the concurrent programming control problem. *IEEE Transactions on Parallel and Distributed Systems*, 22(6):1056–1063, 2011. doi:10.1109/TPDS.2010.172.
- [AS85] Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985. doi:10.1016/0020-0190(85)90056-0.
- [BDH16] Peter A. Buhr, David Dice, and Wim H. Hesselink. Dekker’s mutual exclusion algorithm made rw-safe. *Concurrency and Computation: Practice and Experience*, 28(1):144–165, 2016. doi:10.1002/cpe.3659.
- [BGK<sup>+</sup>19] Olav Bunte, Jan Friso Groote, Jeroen J. A. Keiren, Maurice Laveaux, Thomas Neele, Erik P. de Vink, Wieger Wesselink, Anton Wijs, and Tim A. C. Willemse. The mCRL2 toolset for analysing concurrent systems. In Tomáš Vojnar and Lijun Zhang, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, TACAS’19, volume 11428 of LNCS, pages 21–39. Springer, 2019. doi:10.1007/978-3-030-17465-1\_2.

- [BL93] James E. Burns and Nancy A. Lynch. Bounds on shared memory for mutual exclusion. *Information and Computation*, 107(2):171–184, 1993. doi:10.1006/inco.1993.1065.
- [CDV09] Flavio Corradini, Maria Rita Di Berardini, and Walter Vogler. Time and fairness in a process algebra with non-blocking reading. In *Theory and Practice of Computer Science*, SOFSEM’09, volume 5404 of LNCS, pages 193–204. Springer, 2009. doi:10.1007/978-3-540-95891-8\_20.
- [Dij62] Edsger W. Dijkstra. Over de sequentialiteit van procesbeschrijvingen (ewd-35), 1962. URL: <https://www.cs.utexas.edu/~EWD/ewd00xx/EWD35.PDF>.
- [Dij65] Edsger W. Dijkstra. Solution of a problem in concurrent programming control. *Communications of the ACM*, 8(9):569, 1965. doi:10.1145/365559.365617.
- [GGS26] Rob J. van Glabbeek, Daniele Gorla, and Myrthe Spronck. Models for: On the notions of bounded bypass, and how to make any deadlock-free mutex protocol satisfy one of them, 2026. doi:10.5281/zenodo.17650539.
- [GLS25] Rob J. van Glabbeek, Bas Luttik, and Myrthe S.C. Spronck. Just verification of mutual exclusion algorithms. In Patricia Bouyer and Jaco van de Pol, editors, *Proceedings 36th International Conference on Concurrency Theory, CONCUR’25*, volume 348 of LIPIcs. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPIcs.CONCUR.2025.17.
- [HKG21] Yousra Hafidi, Jeroen J.A. Keiren, and Jan Friso Groote. Fair mutual exclusion for N processes. In *International Conference on Software Testing, Machine Learning and Complex Process Analysis*, pages 149–160. Springer, 2021. doi:10.1007/978-3-031-50423-5\_14.
- [Kes82] Joep L.W. Kessels. Arbitration without common modifiable variables. *Acta Informatica*, 17:135–141, 1982. doi:10.1007/BF00288966.
- [Knu66] Donald E. Knuth. Additional comments on a problem in concurrent programming control. *Communications of the ACM*, 9(5):321–322, 1966. doi:10.1145/355592.365595.
- [Koz83] D. Kozen. Results on the propositional  $\mu$ -calculus. *Theoretical Computer Science*, 27:333–354, 1983. doi:10.1007/BFb0012782.
- [Lam86a] Leslie Lamport. The mutual exclusion problem: part II - statement and solutions. *J. ACM*, 33(2):327–348, 1986. doi:10.1145/5383.5385.
- [Lam86b] Leslie Lamport. On interprocess communication. part II: algorithms. *Distributed Computing*, 1(2):86–101, 1986. doi:10.1007/BF01786228.
- [Pet81] Gary L. Peterson. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3):115–116, 1981. doi:10.1016/0020-0190(81)90106-X.
- [Pra81] Vaughan R. Pratt. A decidable mu-calculus. In *22nd Annual Symposium on Foundations of Computer Science*, pages 421–427. IEEE Computer Society, 1981. doi:10.1109/SFCS.1981.4.
- [Ray13] Michel Raynal. *Concurrent Programming: Algorithms, Principles, and Foundations*. Springer, 2013. doi:10.1007/978-3-642-32027-9.
- [RP76] Ronald L. Rivest and Vaughan R. Pratt. The mutual exclusion problem for unreliable processes: Preliminary report. In *17th IEEE Symposium on Foundations of Computer Science*, pages 1–8, 1976. doi:10.1109/SFCS.1976.33.
- [SGG12] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts*. Wiley Publishing, 9th edition, 2012.
- [SL23] Myrthe S. C. Spronck and Bas Luttik. Process-algebraic models of multi-writer multi-reader non-atomic registers. In Guillermo A. Pérez and Jean-François Raskin, editors, *Proceedings 34th International Conference on Concurrency Theory, CONCUR 2023*, volume 279 of LIPIcs. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi:10.4230/LIPIcs.CONCUR.2023.5.
- [SWPL11] Cheng Shao, Jennifer L. Welch, Evelyn Pierce, and Hyunyoung Lee. Multiwriter consistency conditions for shared memory registers. *SIAM Journal on Computing*, 40(1):28–62, 2011. doi:10.1137/07071158X.
- [Szy88] Boleslaw K. Szymanski. A simple solution to lamport’s concurrent programming problem with linear wait. In *Proc. of the 2nd International Conference on Supercomputing*, page 621–626. ACM, 1988. doi:10.1145/55364.55425.
- [Szy90] Boleslaw K. Szymanski. Mutual exclusion revisited. In *Proc. of the 5th Jerusalem Conference on Next Decade in Information Technology*, pages 110–117, 1990. doi:10.1109/JCIT.1990.128275.
- [Tau06] Gadi Taubenfeld. *Synchronization Algorithms and Concurrent Programming*. Prentice Hall, 2006.